

J 组解析

选择题

第 1 题 C

本题考查新闻。

选自今年二月发布的公告 <https://www.noi.cn/gynoi/tlgd/2025-02-13/837978.shtml>
关于参加 CSP-J/S 认证需符合年龄条件的公告。

第 2 题 D

本题考查变量类型。

一个典型的变量声明语句为 变量类型 变量名;。

在 C++ 中, long long, double, int 都是基本数据类型, 而 define 不是。

第 3 题 C

本题在具体情境中考查编译相关的概念。

在情境中, game.cpp 是源代码, 属于高级语言。

DEV-C++ 是 IDE, 它不仅可以用于编辑代码 (指编辑 game.cpp), 还内置了编译器, 可以把 game.cpp 编译成可执行文件 game.exe。

平时我们说的“运行这个代码”, 实际上运行的是经过编译生成的可执行文件。我们只要发送这个可执行文件就行了。

注意“快捷方式”本身只是一个“地址”, 并不包含程序本身, 所以拷进 U 盘里没什么用。

第 4 题 B

A. $x \& 7 == 0$: 注意 $==$ 的优先级高于 $\&$, 所以需要加括号。

B. $(x \& -x) > 7$: $x \& -x$ 的作用是求出 x 的最低二进制位, 只要判断其是否 > 7 即可。

C. $(x7) == 0$: 不难验证这个命题不可能成立, 因为 $x \mid 7$ 要么是负数, 要么至少是 7。

D. $x \%(2^3) == 0$: 注意这里的 $\wedge$ 不是乘方, 是异或。

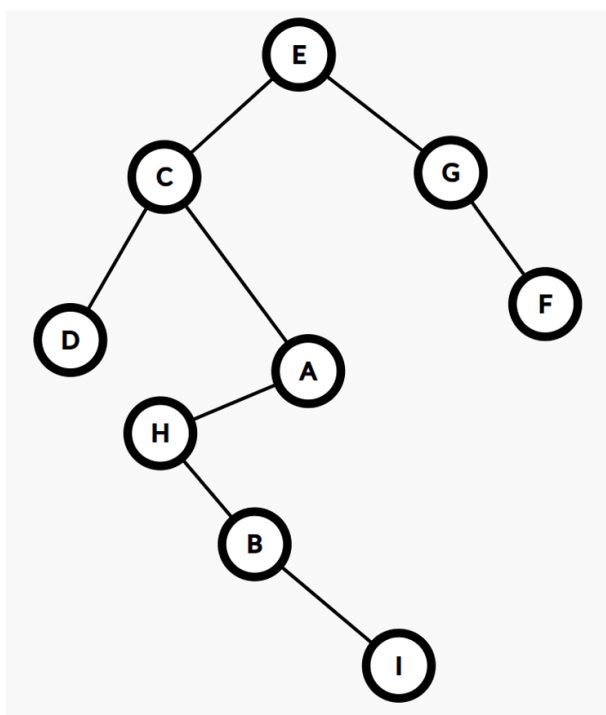
第 5 题 B

本题考查高精度计算中，位数估计的问题。

- A. 考虑 $11 > 9$ ，但是 $1 < 9$ 。
- B. 如果最高位进位那么是 $n + 1$ 位数，否则是 n 位数。
- C. 考虑 $100 - 91 = 9$ ，直接从三位数变成一位数了。
- D. xy 应该是 $n + m$ 位数或者 $n + m + 1$ 位数。

第 6 题 A

本题考查二叉树的遍历。相信大家做过很多类似的题，树长这样。



第 7 题 D

本题考查选手模拟自然语言描述的算法的能力。

- A. $w_2 = 3, v_2 = 8$: 两个物品都装得进去，卡不掉。
- B. $w_2 = 6, v_2 = 13$: 贪心装进物品 2，正解也是物品 2，卡不掉。
- C. $w_2 = 8, v_2 = 18$: 贪心装进物品 2，正解也是物品 2，卡不掉。
- D. $w_2 = 7, v_2 = 12$: 贪心装进物品 1，正解是物品 2，成功。

第 8 题 D

本题考查栈及其 STL 实现。

栈是先进先出（FIFO）结构。只要支持从一侧加入或删除一个元素即可。

`vector` 和 `list` 都支持 `push_back()` 和 `pop_back()` 且都是 $O(1)$ 。

`stack` 本来就是栈。

`queue` 本身是队列，不支持从末尾弹出元素，只能暴力删光了再输出最后一个元素，再把前面的元素加回来。

第 9 题 B

本题考查前缀和与差分。这是 2025 年新增知识点。

如果我们规定 S_i 是 a_i 前缀和， d_i 是 a_i 差分，那么：

$$S_i = S_{i-1} + a_i$$

求前缀和时 `a[i-1]` 应当已经被替换成前缀和，所以 $i-1$ 必须比 i 先访问，从而是正向循环。

$$d_i = a_i - a_{i-1}$$

求差分时 `a[i-1]` 应当还没有被替换成差分，所以 $i-1$ 必须比 i 后访问，从而是反向循环。

第 10 题 A

本题考查分类讨论和计数原理。

分类讨论：

- 所有三位数共 1000 个，其中 125 个三位数可倒置，一共有 875 个不可倒置三位数。

- 可倒置的三位数中，十位倒置后必须和原来一样，有 3 种选法；百位可以任选，共 5 种选法；十位和百位都确定后，个位唯一确定（必须是百位的倒置），因此倒置后和原来相同的三位数共有 $5 \times 3 = 15$ 个。

合计 890 个安全的三位数。

第 11 题 A

本题考查进制转换，需要有一点点的计算功底。

如果不确定自己到底哪一步算错了，可以使用 Windows 计算器中的“程序员”模式辅助检查。

第 12 题 C

本题考查后缀表达式的求值。

对应的中缀表达式为 $(2 * 4 - 7) * (1 + 2)$ ，计算可得 3。

第 13 题 B

本题考查冒泡排序。

以从前往后冒泡为例，从后往前的答案是相同的。

- 第一轮冒泡后，序列变成 0, 2, 2, 0, 5, 9, 0, 2，发生了 1 次。
- 第二轮冒泡后，序列变成 0, 2, 0, 2, 5, 0, 2, 9，发生了 0 次。
- 第三轮冒泡后，序列变成 0, 0, 2, 2, 0, 2, 5, 9，发生了 0 次。
- 第四轮冒泡后，序列变成 0, 0, 2, 0, 2, 2, 5, 9，发生了 1 次。
- 第五轮冒泡后，序列变成 0, 0, 0, 2, 2, 2, 5, 9，发生了 0 次。

合计发生 2 次。

第 14 题 D

本题考查素数与合数的概念。

如果你在考试，注意到 11 确实不能写成两个合数之和（因为最小合数是 4，但 $4+7$ 和 $5+6$ 都不符合题意），而它已经是最大的选项，所以直接选上去就行了。

当然证明其实也不难——尝试证明如果 $n \geq 12$ ，那么 n 总是能写成 $2a+3b, a, b \geq 2$ 的形式即可，读者稍加枚举就可以得到。

第 15 题 A

本题考查树的性质、贪心和排列组合。

首先考虑要让距离和达到最大值，这棵树应该长什么样。

如果这棵树有一个分叉，那么把其中一棵子树掰下来，沿着结点数最少的那个子树方向移动，得到的距离和肯定更大。因此，距离和达到最大值时，这棵树肯定是一条链。

我们不妨假设这条链是 $1, 2, \dots, 2025$ 。对于两个结点 $x, y (x \leq y)$ ，它们的距离等于“满足 $x \leq z < y$ 的结点 z 数量”。

这个求和并不是很好求，于是我们尝试转化成组合数能求的形式。令 $t = x - 1$ ，那么问题就变成了“从 $0, 1, 2, \dots, 2025$ 中选出三个数 $t < z < y$ 的方案数”，这一下就好办——这就是 2026 个数中选 3 个，答案是 $\frac{2026 \times 2025 \times 2024}{6} = 1013 \times 675 \times 2024$ ，模 7 得 1。

阅读题 T1

【分析】

我们剖析这个代码是在做什么事情：

```

1 for(int i = 0 ; i < k ; i++)
2     if(a[i] == 1 || (a[i] != 2 && a[i] % 2 == 0) || a[i] == 9){
3         printf("%d\n", a[i]);
4         return 0;
5     }

```

如果输入（明显是一个多位数）中有某一位为 0,1,4,6,8,9 中的一个，就立刻输出该位并结束。同时注意扫描是从 0 开始递增到 $k-1$ 。

```

1 for(int i = 1 ; i < k ; i++)
2     if(a[i] == 2 || a[i] == 5){
3         printf("%d%d\n", a[i-1], a[i]);
4         return 0;
5     }

```

如果输入中除最高位外（注意是从 1 开始枚举的）某一位为 2 或 5，就输出最高位和该位。

```

1 for(int i = 0 ; i < k ; i++){
2     if(c[a[i]]){
3         printf("%d%d\n", a[i], a[i]);
4         return 0;
5     }
6     c[a[i]] = 1;
7 }

```

如果某个数字出现了 2 次，则输出两次该数字。

```

1 for(int i = 1 ; i < k ; i++)
2     if((a[0] == 2 || a[0] == 5) && a[i] == 7){
3         printf("%d%d\n", a[0], a[i]);
4         return 0;
5     }

```

特判，发现输出的是 27 或 57。但一定会保留最高位。

我们发现，输出的一定是非质数。解答几个误区：

1. 可能输出 1: 1 的确是非质数。
2. 如果有两个 1 会不会输出 11: 第一个循环的讨论就 `return 0` 了。
3. 为什么最后一次一定会保留最高位: 如果最高位不是 2 或 5，一定存在前面已经覆盖过的情况了。

> 输入一个长度为 k 的多位数，删除其中某些位数，使得剩下的位数按原次序排列能得到一个非质数。

[illegible]

【分析】

这个函数并没有什么难分析的点，主要是一个 lambda 函数，不过即使不清楚其含义也不难猜出：

```
1  auto f = [&](int p, int q) {
2      if (p > q) return p - q;
3      else return q - p;
4  };
```

其余部分 $O(n)$ 模拟即可，手算量很小，最多也就 $n = 5$ 而已。

1. 若输入数组 `a` 中所有元素均为 0，则程序输出的 `ans` 显然为 0。故正确。
2. 若将第 14 22 行的循环改为从 `i=1` 开始到 `i<=n`，程序输出结果当然会改变。原因是 `vector` 下标默认从 0 开始。故错误。
3. 当输入为 `4\n2025 2025 -2025 -2025` 时，输出为 8100。故选 D。

4. 当输入为 5\n1 2 3 4 5 时，输出为 16。故选 C。
5. 若将第 16 行改为 $x = x + 2 * a[i];$ ，输入 3\n3 1 2 时，输出为 18，照例是计算题，故选 D。

阅读 T3

【分析】

我们在做的实际上是这样一道题：

- > 你现在有 n 个整数 len （均在 $1 \sim 9$ 范围内），每次你可以进行如下操作：
- > 从中挑选一个偶数 A ；
- > 将其删除，并获得两个 $\frac{A}{2}$ 。
- > 你有相互独立的 q 次询问，请你判断能否通过若干次操作，使得你拥有的数的乘积为 $2^a \times 3^b \times 5^c \times 7^d$ 。若能则输出 1，否则输出 0。

考虑其解法：

- 1 是没用的；
- 2 可以贡献给 a ，也可以拆成两个 1 作废；
- 3 只能贡献给 b （5,7 同理）；
- 4 不管是拆成两个 2 还是拆成四个 1 作废（8 同理）；
- 9 只能贡献给 b 。

关键在 6，有两种处理情况。我们的判断方式是：

- 优先判断 c, d ；
- 得出可以得到 3 的个数的范围，再判断 b 是否处于这个范围；
- 先判断出凑出 3 需要拆开多少个 6；
- 再得出可以得到 2 的个数的范围；
- 随后再判断 a 是否处于这个范围内就行。

【解答】

1 2 FT

3 6 BCDC

1. 尽管算法核心部分时间复杂度的确是 $O(1)$ ，但是输入部分时间复杂度是 $O(n)$ 。

2. 发现 $\text{max_2} - \text{min_2} = \text{cnt}[2] + 2 * \text{cnt}[4] + 4 * \text{cnt}[8] = 5$ 。
3. 对于 c 和 d ，只能用 10 个 5 和 10 个 7 来搞定。对于 a 和 b ，容易发现，最优的解法之一是 3 个 8（共最多可贡献 12 个 2，绰绰有余）和 5 个 9。故 n 最小为 28。
4. 发现一个数最多只能贡献 4 个（即，8 可以贡献 4 个 2 的乘积），故 λ 最小值为 4。
5. 首先， $\text{min_3} = 4$ ， $\text{max_3} = 8$ 。分类讨论：

b	min_2	min(9, max_2)	种数
4	4	9	6
5	3	9	7
6	2	9	8
7	1	9	9
8	0	9	10

合计 40

故所求概率为 $\frac{40}{10 \times 10} = 0.40$ 。

6. 因为 $b = 6$ ，所以至少需要有 3 个 6，至多有 6 个 6。

- 3 个 6: 解二元一次方程组易得 len 只能为 $[2, 4, 4, 4, 4, 4, 6, 6, 6]$ 。所以 $\text{min_2} = 0$ （3 个 6 都要拆）， $\text{max_2} = 11$ 。此时 a 共有 12 种取值。
- 4 个 6: 易得 len 只能为 $[2, 2, 4, 4, 4, 6, 6, 6, 6]$ 。所以 $\text{min_2} = 2$ ， $\text{max_2} = 10$ （只能拆 2 个 6）。此时 a 共有 9 种取值。
- 5 个 6: 易得 len 只能为 $[2, 2, 2, 4, 6, 6, 6, 6, 6]$ 。所以 $\text{min_2} = 4$ ， $\text{max_2} = 9$ （只能拆 1 个 6）。此时 a 共有 6 种取值。
- 6 个 6: len 无解。

因此共有 $12 + 9 + 6 = 27$ 种输入。

补全 T1

解释几个点：

- `int to_t_o(char c1, char c2)`: 目的很明显吧，结合题目意思与给出的代码片段，明显是把两位的 16 进制数转换为 10 进制。那么为什么不能是 `char`? 因为 `char` 的上限是 127，可能会溢出。
- `void to_t_t(int n)`: 看得出来，目的明显是把 10 进制数转换为 16 进制并输出（当然限制是两位）。

- 第 4 题，应填

- `for(int i = 1 ; i <= 7 ; i++)`: 注意 6 位被存储在 0 ~ 5 上。这里满足 `i % 2 == 1` 的有 $i = 1, 3, 5, 7$ ，要跑四遍且有空号，不行。
- `for(int i = 0 ; i < 6 ; i++)`: 这里满足 `i % 2 == 1` 的有 $i = 1, 3, 5$ ，可以。
- `for(int j = 0 ; j < 6 ; j++)`: 这里直接用 'j' 了，会 CE。
- `for(int i = 0 ; i < 6 ; i += 2)`: 这里没有满足 `i % 2 == 1` 的，因为步长为 2，不行。

补全 T2

我们来做这道题。

Q: 为什么一定有解?

A: 考虑把所有数全部转移到比如说 a_1 ，最终 $a=[K,0,0,\dots,0]$ ，取 $x=K$ 即可。

思路 1（仅供参考）

先分解质因数， x 的取值只可能是 S 的质因数。

然后对每个质因数进行操作，即，使每个数变成 0 或者质因数的 k 倍。

先对每堆石头进行处理，处理为该堆石头的数量与它前一个质因数倍数的差值。（当这个数小于质因数，则算它与 0 的差值。）

比如: $[1,2,3,4,5]$ ， $S=15$ ，当处理到质因数为 3 的时候，预处理为 $[1,2,0,1,2]$ （这里的 0 意味着已经是质因数的 k 倍，就可以不存了），然后贪心一下，对预处理好的数组从大到小排序。

这样我们对预处理好的数组做一下前缀和处理，当某一个时刻，前缀和 = 后缀个数 * 质因数 - 后缀和的时候，前缀和即为当前答案。

这里怎么理解呢？其实是：前面的数是用来往外拿，去补后面的数。那么后缀个数 * 质因数 - 后缀和也就是把后缀的石堆补到质因数的 $(k+1)$ 倍。（之前预处理的是它与质因数 k 倍的距离）

思路 2

记 $S = a_1 + a_2 + \dots + a_n$ 。

每次操作不改变所有数的总和 S ，仅重新分配数值。因此，最终所有数均为 x 的倍数时， x 必须是 S 的因子（即 $x \mid S$ ）。

若 x 是合数（如 $x=4$ ），则其质因子（如 $x'=2$ ）必然也是 S 的因子。

将问题转化为 x' 的倍数所需的操作次数不会多于 x 的倍数。因此只需枚举 S 的所有质因子作为候选 x 。

若 S 本身是质数，则唯一可行的 $x=S$ 。此时最小操作次数为 $S - \max\{a_i\}$ （将所有数集中到最大值上）。

对 S 的每个质因子 x ，计算将所有 a_i 变为 x 的倍数的最小操作次数：

1. **余数处理** - 计算每项的余数 $b_i = a_i \bmod x$ ，并保留非零余数（ $b_i = 0$ 无需调整）。 - 所有非零余数的和 $T = \sum b_i$ 必须是 x 的倍数（否则该 x 无解）。
2. **贪心策略：双指针调整余数 - 排序余数**：将非零余数数组 b 从小到大排序。
 - **双指针调整**：
 - 左指针 l 指向最小余数，右指针 r 指向最大余数。
 - 每次操作中，从 b_l 移出 $\min(b_l, x - b_r)$ 到 b_r ：
 - b_l 减少该值， b_r 增加该值（若 b_r 达到 x 则归零）。
 - 累计操作次数为移出的数值。
 - 更新指针：若 $b_l = 0$ 则 l 右移；若 $b_r \geq x$ 则 r 左移。
 - **正确性**：小余数补充大余数至 x 的倍数，可最小化操作次数。
3. **操作次数的计算** - 最终操作次数为所有移出值的总和，即 $cur = \sum \min(b_l, x - b_r)$ 。

复杂度： $O(\sqrt{S} + kn \log n)$ （ k 为质因子数）。

例如：设 $a = [3, 5, 7]$, $S = 15$ ，质因子 $x=3$ 或 5 ：

- $x=3$ ：余数 $b = [0, 2, 1] \rightarrow$ 非零余数 $[1, 2]$ 。
- 排序后 $b = [1, 2]$ ，左指针 $l=0$ ，右指针 $r=1$ 。
- 移 $\min(1, 3-2)=1$ 到 b_r ： $b_l = 0, b_r = 3$ （归零）。
- 操作次数 = 1。
- $x=5$ ：余数 $b = [3, 0, 2] \rightarrow$ 非零余数 $[2, 3]$ 。
- 移 $\min(2, 5-3)=2$ 到 b_r ： $b_l = 0, b_r = 5$ （归零）。
- 操作次数 = 2。
- 最终答案： $\min(1, 2) = 1$ 。

本题代码以 **思路 2** 为基础写成。